CS-433 Machine Learning: Project 1

Jiajun Shen, Joy Liu, Yizhi Zhao
October 2025, EPFL

I. Introduction

This report will discuss the data preprocessing, accuracy metrics, and machine learning methods applied to the Behavioral Risk Factor Surveillance System (BRFSS). In particular, we implement a neural network to accurately predict new cases. Our goal is to predict whether an individual is likely to develop cardio-vascular disease (CVD) based on clinical and lifestyle factors.

II. Preliminary Models

Six basic models were implemented: gradient descent, stochastic gradient descent, least squares regression using normal equations, ridge regression, logistic regression, logistic regression, and regularized logistic regression. While the accuracy of predictions appeared high, the corresponding F1-score was low. This motivates us to gain a better understanding of the data and underlying characteristics.

III. Data Profile and Cleaning

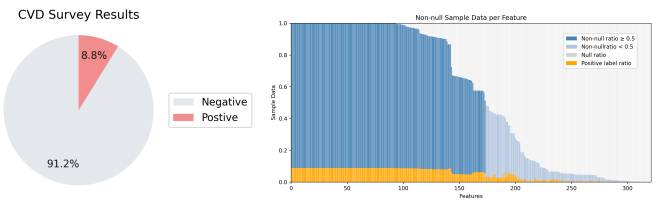
The BRFSS dataset includes a wide range of demographic, behavioral, and medical variables. Each record represents one survey respondent, while each column corresponds to a specific attribute. The target variable MICHD is binary:

$$\text{MICHD} = \begin{cases} 1, & \text{if the respondent has been diagnosed with} \\ & \text{coronary heart disease,} \\ -1, & \text{otherwise.} \end{cases}$$

This setup defines a binary classification problem. A major challenge of this dataset is the strong imbalance between positive and negative labels. As shown in Figure 1a, respondents are overwhelming classified as negative (no MICHD).

Another challenge lies in missing or incomplete values. Several features contain null or undefined values, reducing the reliability of the dataset, as shown in Figure 1b.

Lastly, the dataset contains variables with different numerical scales and measurement units. For example, age is measured in years, BMI is continuous, and categorical survey responses are encoded numerically. Such scale differences can distort model training, particularly for distance-based or gradient-based algorithms.



(a) Positve vs Negative (b) Non-Null Ratio Per Feature

Figure 1: Data Visualization

Based on these challenges, we take the following steps to clean and standardize the data. First, the ID columns are removed as they do not contain predictive information. Missing values are replaced with zero to ensure numerical consistency. Features with null value ratios over 0.5 are dropped due to the irregularity of positive labels. All features were standardized using z-score normalization to stabilize training. We convert the labels into binary form, assigning 0 to non-positive values and 1 to positive values. These steps show an improvement in data quality and stability in model training.

IV. Neural Network

Linear regression often fails to achieve satisfactory performance on nonlinear data. It is also not extendable, as any ensemble of simple linear models remains fundamentally linear (a consequence of simple algebra). To capture complex relationships and introduce nonlinearity into the model, we employ neural networks.

A *neural network* is a parametric model that approximates a function $F : \mathbb{R}^n \to \mathbb{R}^m$ through a composition of linear transformations and nonlinear activations. Formally, for L layers, the output $\hat{y}$ is given by

$$\hat{y} = F(x;\theta) = f(W_L(\cdots f(W_1 x + b_1)\cdots) + b_L),$$

where W_l and b_l are the weight matrix and bias vector at layer l, and $f(\cdot)$ is a nonlinear activation function.

Training minimizes a loss function $\mathcal{L}(y,\hat{y})$, by adjusting parameters $\theta = \{W_l, b_l\}_{l=1}^{L}$ using gradient-based optimization:

$$\theta \leftarrow \theta - \gamma\, \nabla_\theta \mathcal{L}(y, F(x;\theta)),$$

The gradients are efficiently computed layer-by-layer. We apply *backpropagation* to perform gradient descent efficiently.

The gradients are computed as follows (a_l is the output of the l-th layer after activation):

$$\delta_l = \begin{cases} \nabla_{\hat{y}} \odot f', & l = L \\ (W_{l+1}^T \delta_{l+1}) \odot f', & l < L \end{cases}, \quad \nabla_\theta \mathcal{L} = \delta_l a_{l-1}^T$$

V. Soft F1-Loss

Given such biased data, the MSE loss is an unfavorable accuracy metric. Minimizing the loss does not necessarily maximize the F1-Score, as one can always predict negative and have over 90% accuracy based on MSE, while the F1 Score is zero since no positive samples are detected. Some ways to solve this are up-sampling or down-sampling, which balances the data in the sampling process. This is easy to do since we can control the ratio of positive and negative samples in each batch.

However, sampling in each batch can also influence the training time, especially for neural networks with large parameters.

So, instead of using any techniques to adjust the preference of learning, we directly change the loss function aiming at maximizing F1-Score.

Formally, for a batch of labels $\{y_i\}_{i=1}^b, y_i \in \{0,1\}$ and the corresponding predictions $\{\hat{y}_i\}_{i=1}^b, \hat{y}_i \in [0,1]$ clipped between 0 and 1, we define TP, FN, FP, TN as shown in Table I.

Label	Prediction	
	Positive ($\hat{y}$)	Negative ($1-\hat{y}$)
Positive (y)	TP = $\sum_{i=1}^b y \cdot \hat{y}$	FN = $\sum_{i=1}^b (1-\hat{y}) \cdot y$
Negative ($1-y$)	FP = $\sum_{i=1}^b \hat{y} \cdot (1-y)$	TN = $\sum_{i=1}^b (1-y) \cdot (1-\hat{y})$

Table I: Matrix of Soft-F1 Score

The final Soft F1-Score is defined as $F = \frac{2TP}{TP+FP+FN}$, and the gradient to the predicted value is

$$\frac{\partial F}{\partial \hat{y}} = 2 \cdot \frac{TP \cdot (1+y) - y(TP+FP+FN) \cdot}{(TP+FP+FN)^2}$$

In linear regression, we use the gradient to the parameter $\frac{\partial F}{\partial w} = x^\top \frac{\partial F}{\partial \hat{y}}$ and investigate how this loss works in the SGD process. We use learning rate $\gamma = 0.01$ and batch size 32 and run 5000 iterations respectively using MSE and Soft F1-Loss, and the result is shown in Figure 2.

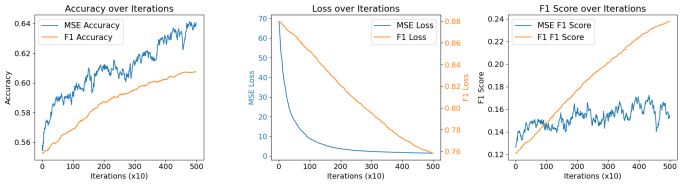


Figure 2: MSE vs Soft F1-Loss

We can see that the model trained on F1-Loss, though has a relatively lower accuracy, present a steadily increasing F1-Score, showing the efficiency of the Soft F1-Loss.

VI. Experiments and Results

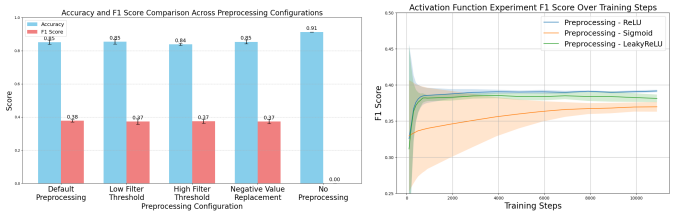
A. Ablation Studies

To better understand how different training settings influence the model performance, we conducted an ablation study focusing on data processing, activation functions, and batch size.

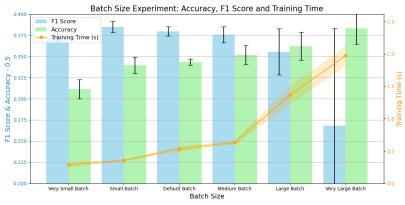
Data preprocessing: In addition to our initial data cleaning, we compared several data preprocessing configurations, including different filtering thresholds and value replacement strategies. As seen in 3a, the model performs consistently across most preprocessing setups. However, without preprocessing, the model achieves a high accuracy but a zero F1-score. This indicates that the classifier predicts all samples as the majority class, likely due to the imbalance in the dataset.

Activation function: We investigated the effect of different activation functions on the model's performance. From the results shown in 3b, we observe that all activation functions exhibit a similar convergence pattern during training, with F1-score increasing steadily before plateauing. ReLU reaches high F1-scores quickly and maintain stable performance, highlighting that ReLU-based activations are the most effective choice in this setting.

Batch sizes: We evaluated the effect of different batch sizes on model performance, with results shown in Figure 3c. The default batch size of 32 provides a well-balanced trade-off between accuracy and F1-score, while maintaining reasonable training time.



(a) Data preprocessing (b) Activation function



(c) Batch size

Figure 3: Ablation studies

B. Final Results

Based on the ablation studies, we selected the optimal configuration that balances model performance and training efficiency. The final settings used for training are as follows:

- **Replacement value:** 0
- **Filtering threshold:** 0.5
- **Batch size:** 32
- **Learning rate:** 0.01
- **Training steps:** 5000
- **Test split:** 20% of the training data for validation

Under this configuration, the model achieved a final performance of Accuracy= 0.8549 ± 0.0017, F1-score= 0.3976 ± 0.0027. The result shows that our chosen model has a great performance in predicting the possibility of CVD.

VII. Summary

In this project, we explored multiple machine learning approaches to predict CVD using a dataset from BRFSS. After implementing machine learning models, we identified and addressed issues caused by imbalanced data by applying systematic data preprocessing, which significantly improved training stability and predictive consistency.

Building on this, we developed a neural network that achieved stronger predictive performance. Through a series of ablation studies, the final configuration achieved an accuracy of 0.8549 ± 0.0017 and an F1-score of 0.3976 ± 0.0027.

Furthermore, we experimented with the Soft F1-Loss as an alternative to the MSE loss. This approach directly optimized the F1-score, improving sensitivity to minority cases and revealing a key trade-off between accuracy and fairness. Overall, this project deepened our understanding of our machine learning methods and the importance of data processing and the trade-off between accuracy and fairness in real-world applications.